```python
#finding second largest number in an array

arr = list(map(int, input("enter numbers: ").split(",")))
if arr[0] > arr[1]:
  max1 = arr[0]
  max2 = arr[1]
else:
  max1 = arr[1]
  max2 = arr[0]
for i in arr[2:]:
  if i > max1:
    max2 = max1
    max1 = i
  elif i > max2:
    max2 = i

print("largest element: ", max1)
print("second largest element: ", max2)
```

```
enter numbers: 1, 3, 10, 15, 13, 21
largest element:  21
second largest element:  15
```

Day 3

Hash tables:

It is defined as data structure that is used to insert, look up, and remove key-value pairs quickly. It operates on the ***hashing concept***, where each key is translated by a hash function into a distinct index in an array. The index functions as a storage location for the matching value.
In simple words, it maps the key with the value.

Applications:

- Hash tables are frequently used for indexing and searching massive volumes of data.
- Data is usually cached in memory via hash tables, enabling rapid access to frequently used info.
- They cab be used to implement database indexes, enabling fast

access to data based on key values.

Hashing:
It is a technique used in data structures that efficiently stores and retrieves data in a way allowing for quick access.

- It involves mapping data to a specific index in a hash table (array of items) using a ***hash function***.
- All three operations (insert, search, delete) can be achieved in ***O(1)*** time on average.

Operation	Average case	Worst case	Reason for worst case
Insert	O(1)	O(n)	Hash collision/ table resizing
Lookup/search	O(1)	O(n)	Extreme hash collision
Deletion	O(1)	O(n)	Long probe sequence caused by high collision density

Dictionaries:
They are used to store data in key:value pairs.

- Each key in a dictionary must be unique.
- Values are accessed using their keys.

```
d = {1: 'Geeks', 2: 'For', 3: 'Geeks'}
print(d[1])
print(d.get(2))
```

Output

```
Geeks
For
```

Sets:
They are unordered and mutable collections used to store unique elements.

- Since they are unordered, elements can't be accessed using indexing.

- Elements are accessed by iteration using a loop.
- Since elements are unordered, order of elements when printed may vary each time code is run.

```
s1 = {"a", "a", "b", "c", "b"}
print(s1)

s2 = {"Geeks", "For", "Geeks"}
for i in s2:
    print(i)
```

Output

```
{'b', 'a', 'c'}
Geeks
For
```

Frequency maps: counting the frequencies using dictionaries

1. The "**Counter**" class from the collections module counts the frequency of each item by creating a dictionary-like object where each unique item becomes a key and its count becomes the value.

```
from collections import Counter
a = ['apple', 'banana', 'apple', 'orange', 'banana', 'banana']
freq = Counter(a)
print(dict(freq))
```

Output

```
{'apple': 2, 'banana': 3, 'orange': 1}
```

2. Using '**get()**' method will count the frequency of each item. It is used to return the current count of an item or '0' if the item is not present in the dictionary yet.

```
a = ['apple', 'banana', 'apple', 'orange', 'banana', 'banana']

freq = {}
for item in a:
    freq[item] = freq.get(item, 0) + 1

print(freq)
```

Output

```
{'apple': 2, 'banana': 3, 'orange': 1}
```

3. **Using dictionary**: this method will count items using a dictionary by checking if the key exists. If it exists, increment; otherwise initialise to 1.

```
a = ['apple', 'banana', 'apple', 'orange', 'banana', 'banana']

freq = {}
for item in a:
    if item in freq:
        freq[item] += 1
    else:
        freq[item] = 1

print(freq)
```

Output

```
{'apple': 2, 'banana': 3, 'orange': 1}
```

Membership checks:

The membership operators test for the membership of an object in a sequence (string, list, tuple).
"**in**" and "**not in**" are the operators.
—> "in" - True if value is found in the sequence
—> "not in" - True if value is not found in the sequence.

```
x = 24
y = 20
my_list = [10, 20, 30, 40, 50]

if (x not in my_list):
    print("x is NOT present in given list")
else:
    print("x is present in given list")

if (y in my_list):
    print("y is present in given list")
else:
    print("y is NOT present in given list")
```

Output

```
x is NOT present in given list
y is present in given list
```

Duplicate detection:

```
#duplicate detection

l = [1, 43, 1, 26, 44, 67, 43, 89, 111, 453, 67]
final_list = []

for i in l:
  if i not in final_list:
    final_list.append(i)

print(final_list)
```

```
[1, 43, 26, 44, 67, 89, 111, 453]
```

Character/element frequency counting:

```
#character/element frequency counting

a = input("enter something: ")
dict = {}

for i in a:
  if i not in dict:
    dict[i] = 1
  else:
    dict[i] += 1

print(dict)
```

```
enter something: 1 2 3 4 5 2 6 7 3
{'1': 1, ' ': 9, '2': 2, '3': 2, '4': 1, '5': 1, '6': 1, '7': 1}
```

```
#character/element frequency counting

a = input("enter something: ")
dict = {}

for i in a:
  if i not in dict:
    dict[i] = 1
  else:
    dict[i] += 1

print(dict)
```

```
enter something: level radar
{'l': 2, 'e': 2, 'v': 1, ' ': 1, 'r': 2, 'a': 2, 'd': 1}
```

Finding common elements in two arrays

```
#finding common elements in two arrays

l1 =input("enter elements for 1st array: ").split()
l2= input("enter elements for 2nd array: ").split()
final_list = []

for i in l1:
  if i in l2:
    final_list.append(i)

print(final_list)
```

```
enter elements for 1st array: 1 2 3 4 5
enter elements for 2nd array: 3 4 5 6 7
['3', '4', '5']
```

Two sum using a dictionary

```
#two sum using a dictionary

l = list(map(int, input("Enter a list of elements: ").split()))
t = int(input("Provide a target value: "))
hash_map = {}

for i, n in enumerate(l):
  diff = t - n
  if diff in hash_map:
    print(hash_map[diff], i)
  hash_map[n] = i
```

```
Enter a list of elements: 21 22 34 45 10
Provide a target value: 55
0 2
3 4
```